

API Health Monitoring System – Documentation

1. Project Overview

This project is a **cloud-native API Health Monitoring System** built using AWS services and Infrastructure as Code (Terraform).

It continuously monitors configured APIs, tracks their health state, detects failures, and records status transitions in a scalable and fault-tolerant manner.

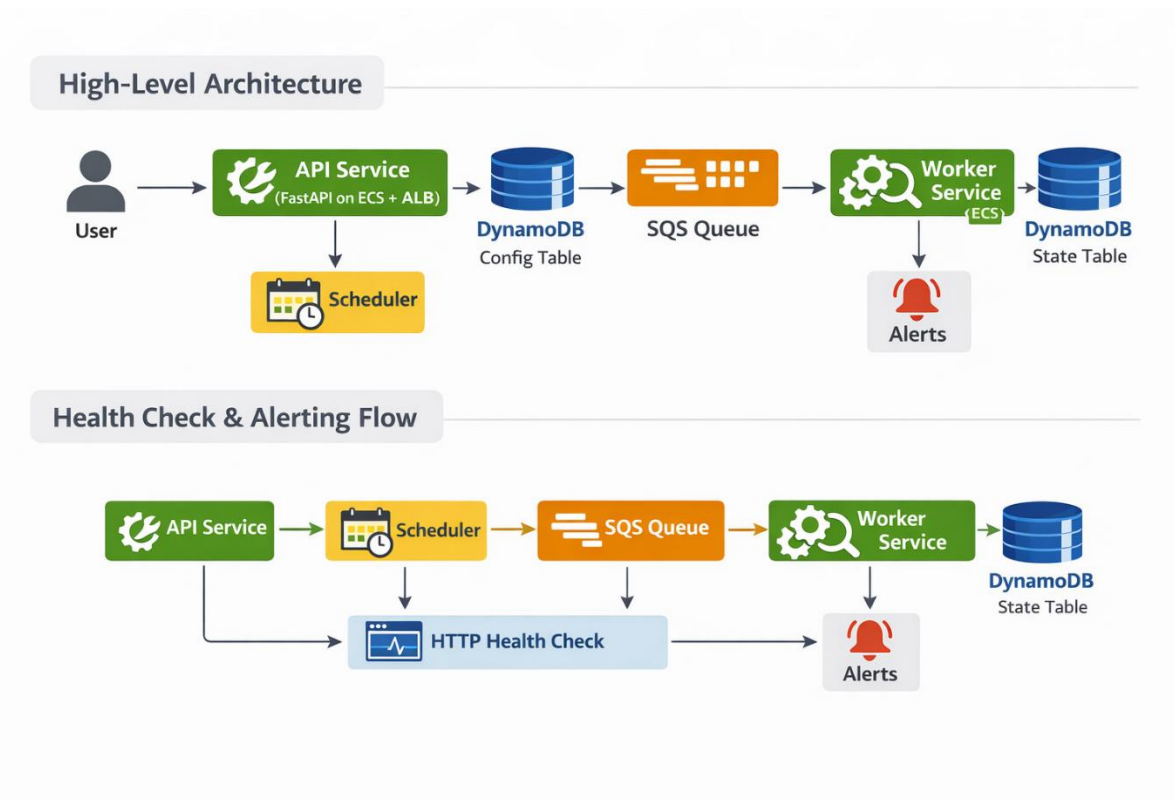
2. Architecture Overview

The system follows a **decoupled, event-driven architecture**.

Core components:

- API Service (FastAPI)
- Scheduler (local job producer)
- Worker Service (ECS Fargate)
- Amazon SQS
- Amazon DynamoDB
- Application Load Balancer
- Amazon ECR
- Terraform (IaC)

Architecture Diagram



3. Application Components & Code Structure

This section explains each application component, its responsibility, and how the project is structured.

3.1 API Service (FastAPI)

The API service is the control plane of the system.

It allows users to register, view, and manage API endpoints that need to be monitored.

Responsibilities:

- Accept user-defined API endpoint configurations
- Store endpoint configurations in DynamoDB

- Expose REST endpoints for scheduler consumption
- Act as a single source of truth for monitored APIs

Key Endpoints:

- POST /endpoints – Register a new API for monitoring
- GET /endpoints – Retrieve all configured endpoints

Deployment:

- Containerized using Docker
- Deployed on Amazon ECS (Fargate)
- Exposed via Application Load Balancer

3.2 Scheduler Application (Local Job Producer)

The scheduler is a lightweight Python service executed locally. It is responsible for periodically triggering health check jobs without performing the checks itself.

Responsibilities:

- Fetch configured endpoints from the API service
- Create stateless health-check jobs
- Push jobs to Amazon SQS at a fixed interval

Design Choice Justification:

- Keeps scheduling logic simple and independent
- Allows workers to scale independently
- Ensures no state is maintained in the scheduler

Message Format Sent to SQS:

```
{  
  "endpoint_id": "uuid",  
  "url": "https://api.example.com",  
  "timeout": 5,  
  "expected_status": 200  
}
```

3.3 Worker Service (ECS Fargate)

The worker service performs the actual health checks.
It is designed to be completely stateless and horizontally scalable.

Responsibilities:

- Poll Amazon SQS for health-check jobs
- Perform HTTP health checks
- Measure latency and status codes
- Detect health state changes
- Persist results in DynamoDB
- Emit alert logs on state transitions

Deployment:

- Containerized using Docker
- Runs as an ECS Fargate service
- Can scale horizontally by increasing task count

3.4 Project Structure & Supporting Files

The project is organized to separate application logic, infrastructure, and configuration.

Key directories:

- **api/** – FastAPI application
- **scheduler/** – Local scheduler job producer
- **worker/** – Health check worker service
- **docker/** – Dockerfiles for API and worker services
- **notifier/** – Alerting logic for health state change notifications (*extensible*)
- **shared/** – Common utilities and shared data models
- **terraform/** – Infrastructure as Code (AWS resources)
- **docs/** – Architecture diagrams and documentation

This structure ensures:

- Clear separation of concerns
- Easy scalability and maintainability
- Infrastructure consistency using Terraform

4. Application Workflow

This section describes how the system operates end-to-end.

4.1 Endpoint Registration

1. A user registers an API endpoint via the API service.
2. Configuration is stored in the health_endpoints DynamoDB table.

3. Each endpoint is assigned a unique identifier.

4.2 Job Scheduling

1. The scheduler periodically calls GET /endpoints.
2. It generates health-check jobs for each endpoint.
3. Jobs are published to the Amazon SQS queue.

4.3 Health Check Execution

1. Worker tasks poll SQS for available jobs.
2. Each worker processes one job at a time.
3. The target API is called using configured timeout and expected status.
4. Latency and response code are captured.

4.4 State Tracking & Change Detection

1. Worker fetches the previous state from DynamoDB.
2. Failure count is incremented on unsuccessful checks.
3. Health state transitions are detected:
 - UNKNOWN → HEALTHY
 - HEALTHY → UNHEALTHY
 - UNHEALTHY → RECOVERED

4.5 Alerting

- Alerts are generated as structured log messages
- Logged to CloudWatch
- Designed to integrate easily with SNS or email in production

Example:

[ALERT] Endpoint xyz changed from HEALTHY → UNHEALTHY

5. Infrastructure Provisioning (Terraform)

All AWS resources are provisioned using **Terraform**, ensuring repeatable and automated infrastructure setup.

Provisioned resources:

- VPC, Subnets, Internet Gateway
- Application Load Balancer
- ECS Cluster & Services (API + Worker)
- ECR Repositories
- SQS Queues (Jobs + DLQ)
- DynamoDB Tables
- IAM Roles & Policies
- CloudWatch Log Groups



Screenshot: Terraform apply success output

```
(venv) aslin@DESKTOP-CBH551J:~/api-health-monitor/terraform$ terraform apply
aws_vpc.main: Refreshing state... [id=vpc-0bf436ea14074fcc]
aws_cloudwatch_log_group.worker_logs: Refreshing state... [id=ecs/api-health-worker]
aws_cloudwatch_log_group.api_logs: Refreshing state... [id=ecs/api-health-api]
aws_ecs_cluster.main: Refreshing state... [id=arn:aws:ecs:ap-south-1:217190816821:cluster/api-health-monitor-cluster]
aws_ecr_repository.api: Refreshing state... [id=api-health-api]
aws_sqs_queue.health_dlq: Refreshing state... [id=https://sqs.ap-south-1.amazonaws.com/217190816821/health-check-dlq]
aws_ecr_repository.worker: Refreshing state... [id=api-health-worker]
aws_dynamodb_table.endpoints: Refreshing state... [id=health_endpoints]
aws_sqs_queue.health_jobs: Refreshing state... [id=https://sqs.ap-south-1.amazonaws.com/217190816821/health-check-jobs]
aws_iam_role.ecs_task_execution: Refreshing state... [id=ecsTaskExecutionRole]
aws_internet_gateway.igw: Refreshing state... [id=igw-02565b88a7c74d0af]
aws_subnet.public: Refreshing state... [id=subnet-08aaa88c15bf647dc]
aws_subnet.public_2: Refreshing state... [id=subnet-004434b6e6788ed7c]
aws_security_group.alb_sg: Refreshing state... [id=sg-0d5bd21f5294f433b]
aws_lb_target_group.api_tg: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:217190816821:targetgroup/api-health-tg/9894c92d125427a4]
aws_route_table.public_rt: Refreshing state... [id=rtb-045b1581d0510d2c5]
aws_security_group.ecs_sg: Refreshing state... [id=sg-08a1f85c57e9c407b]
aws_lb_api_alb: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:217190816821:loadbalancer/app/api-health-alb/d5f74e4de9a74d17]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-056dc4fca0ee494a6]
aws_route_table_association.public_assoc_2: Refreshing state... [id=rtbassoc-0e218b6f6933c4aa8]
aws_lb_listener.api_listener: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:217190816821:listener/app/api-health-alb/d5f74e4de9a74d17/cd824e54777ac97]
aws_iam_role_policy_attachment.ecs_execution_policy: Refreshing state... [id=ecsTaskExecutionRole-20260118083953558900000001]
aws_ecs_task_definition.worker: Refreshing state... [id=api-health-worker]
aws_ecs_task_definition.api: Refreshing state... [id=api-health-api]
aws_ecs_service.worker: Refreshing state... [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/worker-service]
aws_ecs_service.api: Refreshing state... [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/api-service]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create
~ update in-place
-/ destroy and then create replacement
```

```
# aws_iam_role_policy_attachment.ecs_task_dynamodb will be created
+ resource "aws_iam_role_policy_attachment" "ecs_task_dynamodb" {
+   id           = (known after apply)
+   policy_arn   = "arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess"
+   role         = "ecsTaskRole"
}

# aws_iam_role_policy_attachment.ecs_task_sqs will be created
+ resource "aws_iam_role_policy_attachment" "ecs_task_sqs" {
+   id           = (known after apply)
+   policy_arn   = "arn:aws:iam::aws:policy/AmazonSQSFullAccess"
+   role         = "ecsTaskRole"
}

Plan: 4 to add, 1 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_ecs_task_definition.worker: Destroying... [id=api-health-worker]
aws_ecs_task_definition.worker: Destruction complete after 1s
aws_iam_role.ecs_task_role: Creating...
aws_iam_role.ecs_task_role: Creation complete after 3s [id=ecsTaskRole]
aws_iam_role_policy_attachment.ecs_task_dynamodb: Creating...
aws_iam_role_policy_attachment.ecs_task_sqs: Creating...
aws_ecs_task_definition.worker: Creating...
aws_ecs_task_definition.worker: Creation complete after 0s [id=api-health-worker]
aws_ecs_service.worker: Modifying... [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/worker-service]
aws_ecs_service.worker: Modifications complete after 1s [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/worker-service]
aws_iam_role_policy_attachment.ecs_task_sqs: Creation complete after 1s [id=ecsTaskRole-20260118100300081800000001]
aws_iam_role_policy_attachment.ecs_task_dynamodb: Creation complete after 1s [id=ecsTaskRole-20260118100300478100000002]

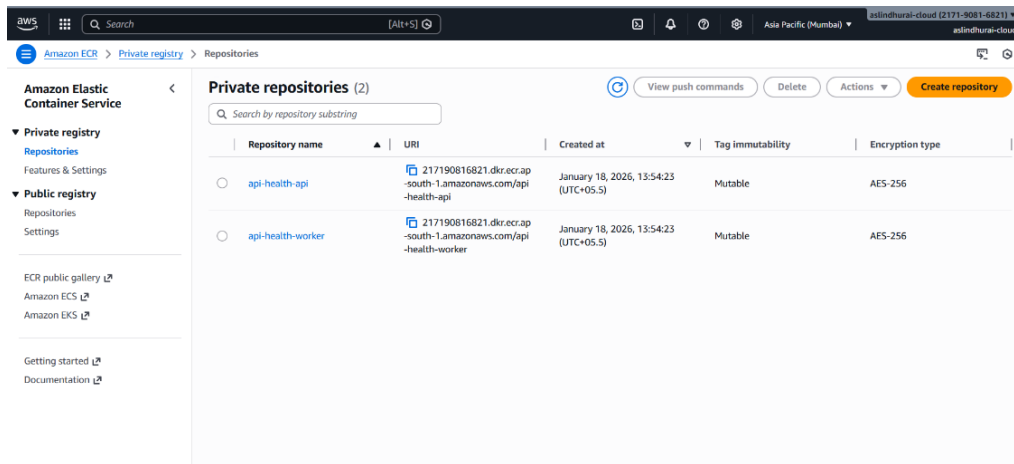
Apply complete! Resources: 4 added, 1 changed, 1 destroyed.
(venv) aslin@DESKTOP-CBH551J:~/api-health-monitor/terraform$
```

6. Container Images (ECR)

Two Docker images are built and pushed to Amazon ECR:

- api-health-api – API service
- api-health-worker – Background worker

Screenshot: ECR repositories page



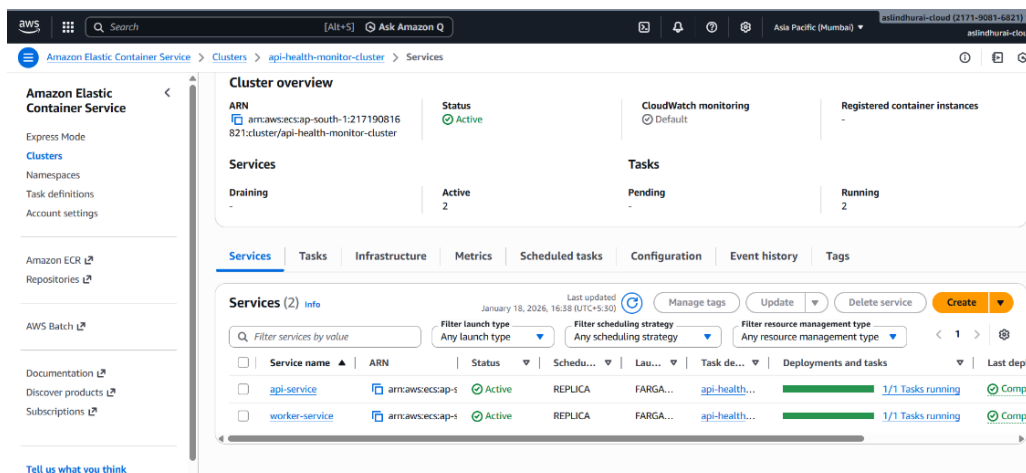
7. ECS Deployment

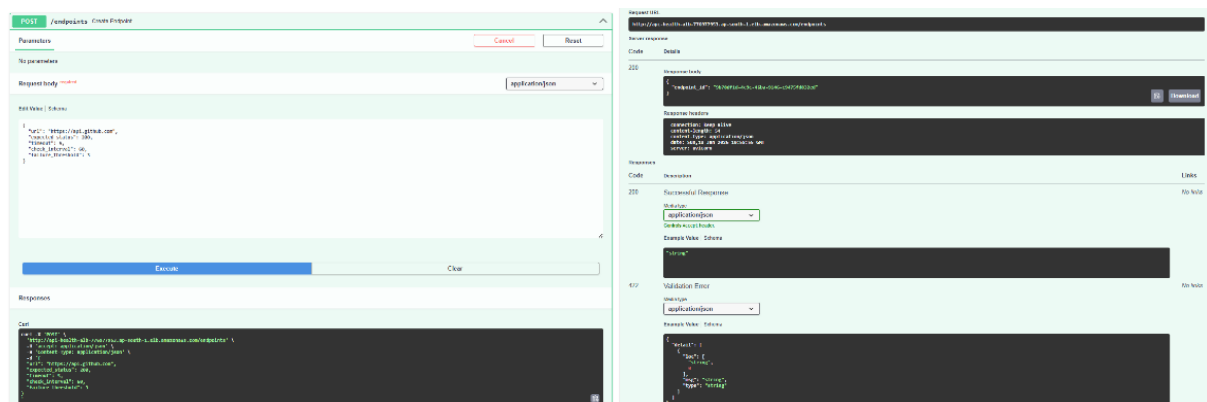
The application runs on **ECS Fargate** for serverless container execution.

ECS Services:

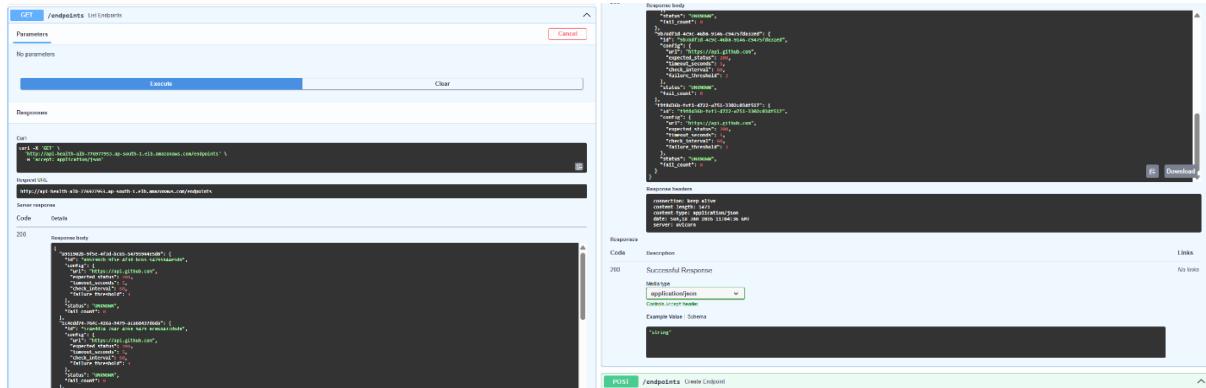
- **api-service**
 - Exposed via Application Load Balancer
- **worker-service**
 - Polls SQS and performs health checks

Screenshot: ECS Cluster → Services (both running)





Screenshot: GET /endpoints response



The screenshot shows a REST client interface with the URL `GET /endpoints`. The response is a JSON array of endpoint configurations. The response body is displayed in a dark-themed editor. The response status is 200, and the response type is `application/json`.

```
{
  "endpoints": [
    {
      "endpoint_id": "12345678901234567890123456789012",
      "url": "https://api.github.com/repos/octocat/Hello-World",
      "timeout": 5,
      "expected_status": 200
    },
    {
      "endpoint_id": "09876543210987654321098765432109",
      "url": "https://api.github.com/repos/octocat/Hello-World",
      "timeout": 5,
      "expected_status": 200
    }
  ]
}
```

9. Scheduler (Local Application)

The scheduler runs **locally** and acts as a **job producer**.

Responsibilities:

- Fetch endpoints from API
- Convert endpoint config to job format
- Push jobs to SQS at regular intervals

Example SQS Message:

```
{
  "endpoint_id": "uuid",
  "url": "https://api.github.com",
  "timeout": 5,
  "expected_status": 200
}
```

Screenshot: Scheduler terminal logs showing job enqueue

```
(venv) aslin@DESKTOP-CBHSSIJ:~/api-health-monitor$ python scheduler/scheduler.py
INFO:botocore.credentials:Found credentials in shared credentials file: ~/.aws/credentials
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
INFO:__main__:Enqueued job for https://api.github.com
```

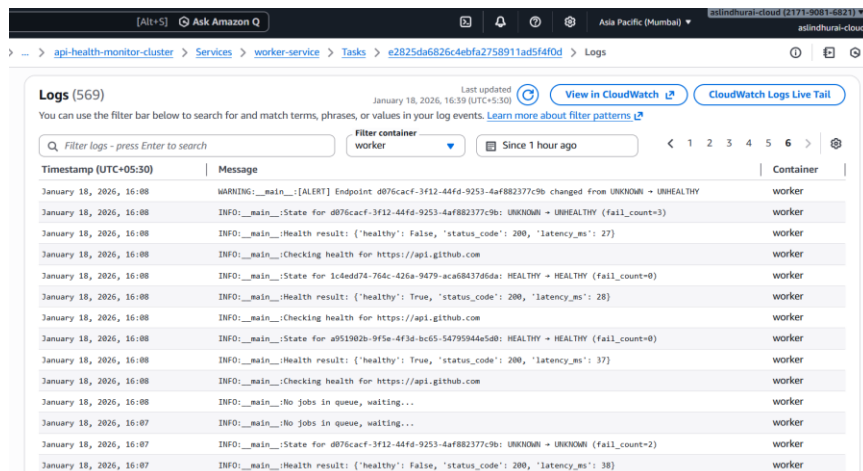
10. Worker Service

The worker runs on ECS and performs health checks.

Responsibilities:

- Poll SQS
- Perform HTTP health check
- Detect state changes
- Persist state to DynamoDB
- Emit alerts via logs

Screenshot: CloudWatch logs – health check results



Timestamp (UTC+05:30)	Message	Container
January 18, 2026, 16:08	WARNING:__main__: [ALERT] Endpoint d076cacf-3f12-44fd-9253-4af8b2377c9b changed from UNKNOWN → UNHEALTHY	worker
January 18, 2026, 16:08	INFO:__main__:State for d076cacf-3f12-44fd-9253-4af8b2377c9b: UNKNOWN → UNHEALTHY (fail_count=3)	worker
January 18, 2026, 16:08	INFO:__main__:Health result: {'healthy': False, 'status_code': 200, 'latency_ms': 27}	worker
January 18, 2026, 16:08	INFO:__main__:Checking health for https://api.github.com	worker
January 18, 2026, 16:08	INFO:__main__:State for 1c4edd74-76dc-426a-9479-aca68437d6da: HEALTHY → HEALTHY (fail_count=0)	worker
January 18, 2026, 16:08	INFO:__main__:Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 28}	worker
January 18, 2026, 16:08	INFO:__main__:Checking health for https://api.github.com	worker
January 18, 2026, 16:08	INFO:__main__:State for a951902b-9f5e-4f3d-bc65-54795944e5db: HEALTHY → HEALTHY (fail_count=0)	worker
January 18, 2026, 16:08	INFO:__main__:Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 37}	worker
January 18, 2026, 16:08	INFO:__main__:Checking health for https://api.github.com	worker
January 18, 2026, 16:08	INFO:__main__:No jobs in queue, waiting...	worker
January 18, 2026, 16:07	INFO:__main__:No jobs in queue, waiting...	worker
January 18, 2026, 16:07	INFO:__main__:State for d076cacf-3f12-44fd-9253-4af8b2377c9b: UNKNOWN → UNKNOWN (fail_count=2)	worker
January 18, 2026, 16:07	INFO:__main__:Health result: {'healthy': False, 'status_code': 200, 'latency_ms': 38}	worker

[Alt+S] Ask Amazon Q			Asia Pacific (Mumbai)			asindhurai-cloud		
api-health-monitor-cluster			Services			worker-service		
Tasks			e2825da6826c4ebfa2758911ad5f4f0d			Logs		
Logs (569)			Last updated January 18, 2026, 16:39 (UTC+5:30)			View in CloudWatch		
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns			CloudWatch Logs Live Tail					
Filter logs - press Enter to search			Filter container worker			Since 1 hour ago		
Timestamp (UTC+05:30)			Message			Container		
January 18, 2026, 16:09			INFO:__main__:Checking health for https://api.github.com			worker		
January 18, 2026, 16:09			WARNING:__main__: [ALERT] Endpoint 36c89c86-fef5-4aa3-8914-8db05aed8624 changed from UNKNOWN → HEALTHY			worker		
January 18, 2026, 16:09			INFO:__main__:State for 36c89c86-fef5-4aa3-8914-8db05aed8624: UNKNOWN → HEALTHY (fail_count=0)			worker		
January 18, 2026, 16:09			INFO:__main__:Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 27}			worker		
January 18, 2026, 16:09			INFO:__main__:Checking health for https://api.github.com			worker		
January 18, 2026, 16:09			INFO:__main__:State for 1c4edd74-764c-426a-9479-aca68437d6da: HEALTHY → HEALTHY (fail_count=0)			worker		
January 18, 2026, 16:09			INFO:__main__:Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 26}			worker		
January 18, 2026, 16:09			INFO:__main__:Checking health for https://api.github.com			worker		
January 18, 2026, 16:09			INFO:__main__:State for a951902b-9f5e-4f3d-bc65-5479594de500: HEALTHY → HEALTHY (fail_count=0)			worker		
January 18, 2026, 16:09			INFO:__main__:Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 257}			worker		
January 18, 2026, 16:09			INFO:__main__:Checking health for https://api.github.com			worker		
January 18, 2026, 16:09			INFO:__main__:No jobs in queue, waiting...			worker		
January 18, 2026, 16:08			INFO:__main__:No jobs in queue, waiting...			worker		

11. State Management (DynamoDB)

Health states are stored in DynamoDB.

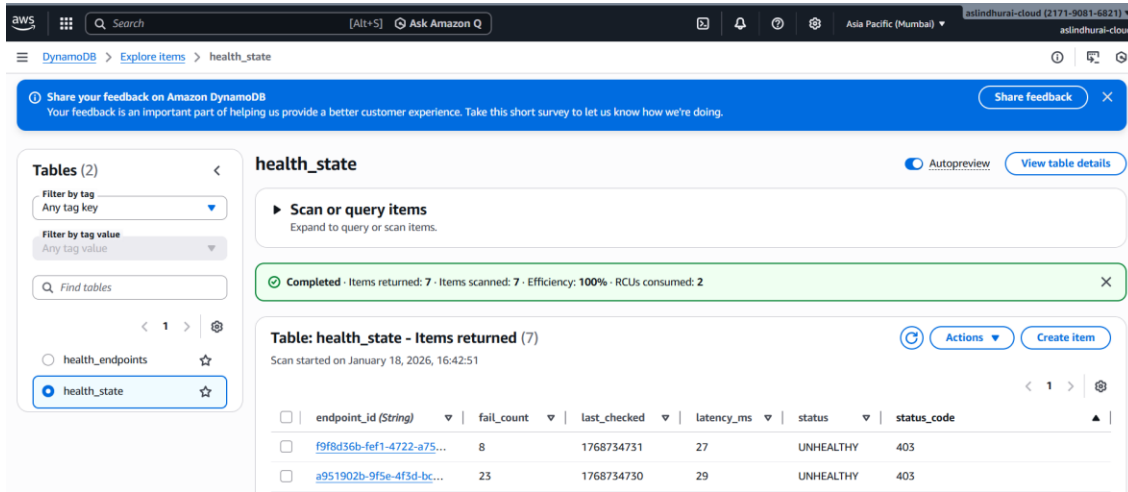
Tables:

- **health_endpoints**
- **health_state**

health_state attributes:

- endpoint_id
- status
- fail_count
- latency_ms
- status_code
- last_checked

Screenshot: DynamoDB health_state table items



The screenshot shows the AWS DynamoDB console interface. On the left, a sidebar lists two tables: 'health_endpoints' and 'health_state', with 'health_state' selected. The main area displays the 'health_state' table details, including a scan status bar indicating 'Completed' with 7 items returned and 100% efficiency. Below this, a table titled 'Table: health_state - Items returned (7)' shows the following data:

endpoint_id (String)	fail_count	last_checked	latency_ms	status	status_code
f9f8d36b-fef1-4722-a75...	8	1768734731	27	UNHEALTHY	403
a951902b-9f5e-4f3d-bc...	23	1768734730	29	UNHEALTHY	403

12. State Change Detection & Alerts

The system detects **health transitions** and logs alerts:

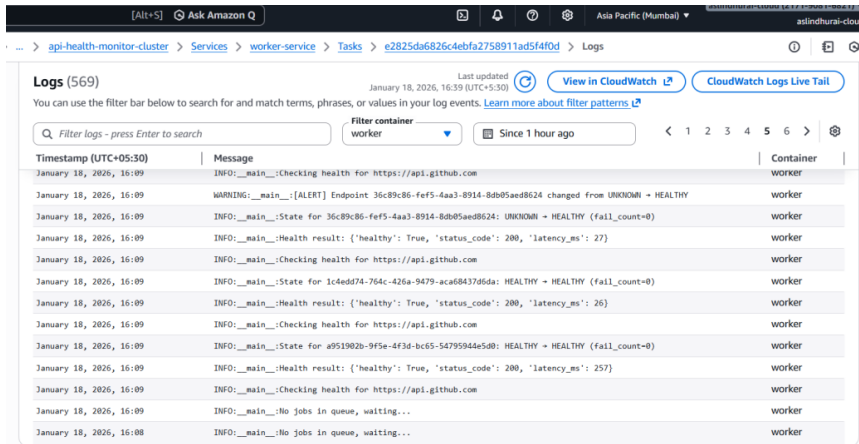
Scenario Alert

First success UNKNOWN → HEALTHY

After failures HEALTHY → UNHEALTHY

Recovery UNHEALTHY → HEALTHY

Screenshot: CloudWatch logs showing [ALERT] messages



The screenshot shows the AWS CloudWatch console interface. The top navigation bar indicates the current view is for the 'worker' container. The logs are filtered by 'worker' and show a sequence of events. The logs are as follows:

Timestamp (UTC+05:30)	Message	Container
January 18, 2026, 16:09	INFO: _main_ Checking health for https://api.github.com	worker
January 18, 2026, 16:09	WARNING: _main_ [ALERT] Endpoint 36c89c86-fef5-4aa3-8914-8db05aed8624 changed from UNKNOWN → HEALTHY	worker
January 18, 2026, 16:09	INFO: _main_ State for 36c89c86-fef5-4aa3-8914-8db05aed8624: UNKNOWN → HEALTHY (fail_count=0)	worker
January 18, 2026, 16:09	INFO: _main_ Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 27}	worker
January 18, 2026, 16:09	INFO: _main_ Checking health for https://api.github.com	worker
January 18, 2026, 16:09	INFO: _main_ State for 1c4ed874-764c-426a-9479-aca68437d6da: HEALTHY → HEALTHY (fail_count=0)	worker
January 18, 2026, 16:09	INFO: _main_ Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 26}	worker
January 18, 2026, 16:09	INFO: _main_ Checking health for https://api.github.com	worker
January 18, 2026, 16:09	INFO: _main_ State for a951902b-9f5e-4f3d-bc65-54795944e5d8: HEALTHY → HEALTHY (fail_count=0)	worker
January 18, 2026, 16:09	INFO: _main_ Health result: {'healthy': True, 'status_code': 200, 'latency_ms': 257}	worker
January 18, 2026, 16:09	INFO: _main_ Checking health for https://api.github.com	worker
January 18, 2026, 16:09	INFO: _main_ No jobs in queue, waiting...	worker
January 18, 2026, 16:08	INFO: _main_ No jobs in queue, waiting...	worker

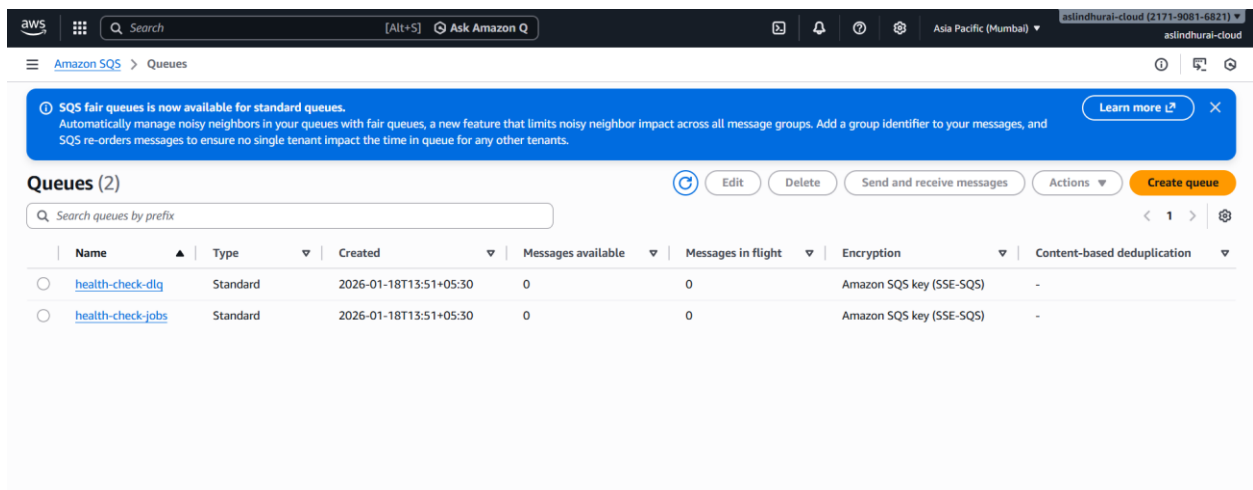
13. Message Queue (SQS)

Amazon SQS decouples the scheduler and worker.

Queues:

- health-check-jobs
- health-check-dlq

Screenshot: SQS queues list



Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
health-check-dlq	Standard	2026-01-18T13:51:05:30	0	0	Amazon SQS key (SSE-SQS)	-
health-check-jobs	Standard	2026-01-18T13:51:05:30	0	0	Amazon SQS key (SSE-SQS)	-

14. Cleanup & Cost Control

All resources can be safely destroyed using:

terraform destroy

Screenshot: Terraform destroy completion

```
(venv) aslin@DESKTOP-CBH55IJ:~/api-health-monitor/terraform$ terraform destroy
aws_ecs_cluster.main: Refreshing state... [id=arn:aws:ecs:ap-south-1:217190816821:cluster/api-health-monitor-cluster]
aws_vpc.main: Refreshing state... [id=vpc-0bf436ea14074fcce]
aws_ecr_repository.api: Refreshing state... [id=api-health-api]
aws_cloudwatch_log_group.api_logs: Refreshing state... [id=ecs/api-health-api]
aws_cloudwatch_log_group.worker_logs: Refreshing state... [id=ecs/api-health-worker]
aws_iam_role.ecs_task_execution: Refreshing state... [id=ecsTaskExecutionRole]
aws_sqs_queue.health_dlg: Refreshing state... [id=https://sqs.ap-south-1.amazonaws.com/217190816821/health-check-dlg]
aws_iam_role.ecs_task_role: Refreshing state... [id=ecsTaskRole]
aws_dynamodb_table.endpoints: Refreshing state... [id=health_endpoints]
aws_dynamodb_table.health_state: Refreshing state... [id=health_state]
aws_ecr_repository.worker: Refreshing state... [id=api-health-worker]
aws_internet_gateway.tgw: Refreshing state... [id=igw-02565b88a7c74d0af]
aws_subnet.public_2: Refreshing state... [id=subnet-004434b6e6788ed7c]
aws_subnet.public: Refreshing state... [id=subnet-08aaa88c15bf847dc]
aws_lb_target_group.api_sg: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:217190816821:targetgroup/api-health-tg/9894c92d125427a4]
aws_security_group.alb_sg: Refreshing state... [id=sg-0d5bd21f5294f433b]
aws_route_table.public_rt: Refreshing state... [id=rtb-045b1581d0510d2c5]
aws_security_group.ecs_sg: Refreshing state... [id=sg-08a1f05c57e9c407b]
aws_lb_api_alb: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:217190816821:loadbalancer/app/api-health-alb/d5f74e4de9a74d17]
aws_route_table_association.public_assoc_2: Refreshing state... [id=rtbassoc-0e218b6f6933c4aa8]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-056dc4fca0ee494a6]
aws_iam_role_policy_attachment.ecs_task_sqs: Refreshing state... [id=ecsTaskRole-20260118083953558900000001]
aws_iam_role_policy_attachment.ecs_task_dynamodb: Refreshing state... [id=ecsTaskRole-20260118083953558900000001]
aws_iam_role_policy_attachment.ecs_execution_policy: Refreshing state... [id=ecsTaskExecutionRole-20260118083953558900000001]
aws_ecs_task_definition.worker: Refreshing state... [id=api-health-worker]
aws_ecs_task_definition.api: Refreshing state... [id=api-health-api]
aws_ecs_service.worker: Refreshing state... [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/worker-service]
aws_lb_listener.api_listener: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:217190816821:listener/app/api-health-alb/d5f74e4de9a74d17/cd8424c94777ac97]
aws_ecs_service.api: Refreshing state... [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/api-service]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  destroy

Terraform will perform the following actions:
```

```
# aws_vpc.main will be destroyed
- resource "aws_vpc" "main" {
  - arn                    = "arn:aws:ec2:ap-south-1:217190816821:vpc/vpc-0bf436ea14074fcce" -> null
  - assign_generated_ipv6_cidr_block = false -> null
  - cidr_block                = "10.0.0.0/16" -> null
  - default_network_acl_id    = "acl-00ce3dc4741cc1985" -> null
  - default_route_table_id    = "rtb-0fedfb9b6acfbcs97" -> null
  - default_security_group_id = "sg-0d36776e668004f32" -> null
  - dhcp_options_id          = "dopt-0aaa2612a6b61020f" -> null
  - enable_dns_hostnames     = true -> null
  - enable_dns_support       = true -> null
  - enable_network_address_usage_metrics = false -> null
  - id                      = "vpc-0bf436ea14074fcce" -> null
  - instance_tenancy         = "default" -> null
  - ipv6_netmask_length      = 0 -> null
  - main_route_table_id      = "rtb-0fedfb9b6acfbcs97" -> null
  - owner_id                 = "217190816821" -> null
  - tags                    = {
    - "Name" = "api-health-monitor-vpc"
  } -> null
  - tags_all              = {
    - "Name" = "api-health-monitor-vpc"
  } -> null
} # (4 unchanged attributes hidden)

Plan: 0 to add, 0 to change, 30 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_iam_role_policy_attachment.ecs_execution_policy: Destroying... [id=ecsTaskExecutionRole-20260118083953558900000001]
aws_cloudwatch_log_group.api_logs: Destroying... [id=ecs/api-health-api]
aws_cloudwatch_log_group.worker_logs: Destroying... [id=ecs/api-health-worker]
aws_iam_role_policy_attachment.ecs_task_sqs: Destroying... [id=ecsTaskRole-20260118083953558900000001]
aws_sqs_queue.health_dlg: Destroying... [id=https://sqs.ap-south-1.amazonaws.com/217190816821/health-check-dlg]
aws_route_table_association.public_assoc: Destroying... [id=rtbassoc-056dc4fca0ee494a6]
aws_dynamodb_table.health_state: Destroying... [id=health_state]
aws_dynamodb_table.endpoints: Destroying... [id=health_endpoints]
aws_sqs_queue.health_jobs: Destroying... [id=https://sqs.ap-south-1.amazonaws.com/217190816821/health-check-jobs]
aws_ecs_service.api: Destroying... [id=arn:aws:ecs:ap-south-1:217190816821:service/api-health-monitor-cluster/api-service]
aws_cloudwatch_log_group.api_logs: Destruction complete after 0s
aws_cloudwatch_log_group.worker_logs: Destruction complete after 0s
```

```
(venv) aslin@DESKTOP-CBH55IJ:~/api-health-monitor/terraform$ terraform destroy
```

No changes. No objects need to be destroyed.

Either you have not created any objects yet or the existing objects were already deleted outside of Terraform.

Destroy complete! Resources: 0 destroyed.

```
(venv) aslin@DESKTOP-CBH55IJ:~/api-health-monitor/terraform$
```

15. Key Learnings & Highlights

- Event-driven architecture
- Stateless workers
- Scalable health monitoring
- Infrastructure as Code
- AWS ECS + SQS + DynamoDB integration
- Clean teardown with zero leftover resources

16. Design Decisions & Trade-offs

Design Trade-offs:

Alerts are currently logged instead of being delivered via email or messaging services to comply with the constraint of avoiding managed monitoring tools. In a production system, this could be extended using SNS or Webhooks. Scheduler is run locally for simplicity, but can be migrated to EventBridge or ECS Scheduled Tasks for full cloud-native operation.